

Data Mining

Lab - 1

Ayushi Patel

23010101199

Introduction to Pandas Library
Function:

Step-1 Import the pandas Libraries

```
In [3]: import pandas as pd
```

Step-2 Import the dataset from this:....

```
In [9]: df=pd.read_csv('titanic.csv')
```

Step-3 Read csv or excel File

```
In [ ]: df=pd.read_csv('titanic.csv')
```

Step-4 Print Data from csv or excel File

```
In [11]: df
```

Out[11]:	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STC 31
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	3
...	...	...	...	...	...	...	...	...	
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	2
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	1
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	1
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	3

891 rows × 12 columns

Step-5 See the First 10 Rows

In [13]: `df.head(10)`

Out[13]:	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Tic
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	21
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/ 3101
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237

Step-6 See the Last 10 Rows

```
In [15]: df.tail(10)
```

Out[15]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
881	882	0	3	Markun, Mr. Johann	male	33.0	0	0	
882	883	0	3	Dahlberg, Miss. Gerda Ulrika	female	22.0	0	0	
883	884	0	2	Banfield, Mr. Frederick James	male	28.0	0	0	C.A.
884	885	0	3	Sutehall, Mr. Henry Jr	male	25.0	0	0	SO
885	886	0	3	Rice, Mrs. William (Margaret Norton)	female	39.0	0	5	
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	

Step-7 Data type of each columns

In [17]: `df.dtypes`

```
Out[17]: PassengerId      int64
Survived      int64
Pclass        int64
Name          object
Sex           object
Age           float64
SibSp         int64
Parch         int64
Ticket        object
Fare          float64
Cabin         object
Embarked      object
dtype: object
```

Step-8 Display Summary Information

```
In [19]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   PassengerId     891 non-null   int64
1   Survived        891 non-null   int64
2   Pclass          891 non-null   int64
3   Name            891 non-null   object
4   Sex             891 non-null   object
5   Age             714 non-null   float64
6   SibSp           891 non-null   int64
7   Parch           891 non-null   int64
8   Ticket          891 non-null   object
9   Fare            891 non-null   float64
10  Cabin           204 non-null   object
11  Embarked        889 non-null   object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

```
In [21]: df.describe()
```

Out[21]:

	PassengerId	Survived	Pclass	Age	SibSp	Parc
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.38159
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.80605
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.00000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.00000
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.00000
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.00000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.00000

Step-9 Access a specific column

In [36]: `df[['Age', 'Fare', 'Name', 'PassengerId']]`

Out[36]:

	Age	Fare	Name	PassengerId
0	22.0	7.2500	Braund, Mr. Owen Harris	1
1	38.0	71.2833	Cumings, Mrs. John Bradley (Florence Briggs Th...	2
2	26.0	7.9250	Heikkinen, Miss. Laina	3
3	35.0	53.1000	Futrelle, Mrs. Jacques Heath (Lily May Peel)	4
4	35.0	8.0500	Allen, Mr. William Henry	5
...	...	...	...	...
886	27.0	13.0000	Montvila, Rev. Juozas	887
887	19.0	30.0000	Graham, Miss. Margaret Edith	888
888	NaN	23.4500	Johnston, Miss. Catherine Helen "Carrie"	889
889	26.0	30.0000	Behr, Mr. Karl Howell	890
890	32.0	7.7500	Dooley, Mr. Patrick	891

891 rows × 4 columns

In [32]: `df.shape[0]`

Out[32]: 891

Step-10 Access rows by their integer location

```
In [48]: # df.iloc[3]
# df.iloc[10:20]
# df.iloc[10:20,0:5]
df.iloc[:,0:5]
# df
```

Out[48]:	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
	0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0
	1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0 PC
	2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0 STC 31
	3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0 1
	4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0 3
	...	...	...	...	...	...	...	...	...
	886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0 2
	887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0 1
	888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2
	889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0 1
	890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0 3

891 rows × 12 columns

Step-11 Delete a specific Column

```
In [54]: df.drop('Age',axis=1) #drop for the particular cell only, temporary delete
```

```
Out[54]:
```

	PassengerId	Survived	Pclass	Name	Sex	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	1	0	A/5 21171
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	1	0	PC 17599
2	3	1	3	Heikkinen, Miss. Laina	female	0	0	STON/O2. 3101282
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	1	0	113803
4	5	0	3	Allen, Mr. William Henry	male	0	0	373450
...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	0	0	211536
887	888	1	1	Graham, Miss. Margaret Edith	female	0	0	112053
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	1	2	W./C. 6607
889	890	1	1	Behr, Mr. Karl Howell	male	0	0	111369
890	891	0	3	Dooley, Mr. Patrick	male	0	0	370376

891 rows × 11 columns

```
In [63]: df.drop('Embarked',axis=1,inplace=True) #always use this command for only or
```

In [65]: df

Out[65]:

	PassengerId	Survived	Pclass	Name	Sex	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	1	0	A/5 21171
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	1	0	PC 17599
2	3	1	3	Heikkinen, Miss. Laina	female	0	0	STON/O2. 3101282
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	1	0	113803
4	5	0	3	Allen, Mr. William Henry	male	0	0	373450
...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	0	0	211536
887	888	1	1	Graham, Miss. Margaret Edith	female	0	0	112053
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	1	2	W./C. 6607
889	890	1	1	Behr, Mr. Karl Howell	male	0	0	111369
890	891	0	3	Dooley, Mr. Patrick	male	0	0	370376

891 rows × 10 columns

In [67]: df.drop(3,inplace=True)

In [69]: df

Out[69]:

	PassengerId	Survived	Pclass	Name	Sex	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	1	0	A/5 21171
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	1	0	PC 17599
2	3	1	3	Heikkinen, Miss. Laina	female	0	0	STON/O2. 3101282
4	5	0	3	Allen, Mr. William Henry	male	0	0	373450
5	6	0	3	Moran, Mr. James	male	0	0	330877
...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	0	0	211536
887	888	1	1	Graham, Miss. Margaret Edith	female	0	0	112053
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	1	2	W./C. 6607
889	890	1	1	Behr, Mr. Karl Howell	male	0	0	111369
890	891	0	3	Dooley, Mr. Patrick	male	0	0	370376

890 rows × 10 columns

Step-12 Create a new Column

In [101]:

```
# df['Amount']=df['Fare']
# df['Amount']=df['Fare']*100
# df['Amount']=5
df['Amount_2'] = df.Fare.apply(lambda x:True if x>10 else False) # to create
df
```

Out[101]...

	PassengerId	Survived	Pclass	Name	Sex	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	1	0	A/5 21171
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	1	0	PC 17599
2	3	1	3	Heikkinen, Miss. Laina	female	0	0	STON/O2. 3101282
4	5	0	3	Allen, Mr. William Henry	male	0	0	373450
5	6	0	3	Moran, Mr. James	male	0	0	330877
...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	0	0	211536
887	888	1	1	Graham, Miss. Margaret Edith	female	0	0	112053
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	1	2	W./C. 6607
889	890	1	1	Behr, Mr. Karl Howell	male	0	0	111369
890	891	0	3	Dooley, Mr. Patrick	male	0	0	370376

890 rows × 12 columns

Step-13 Perform Condition Selection on DataFrame

```
In [99]: # df.query('Fare>10')
df.query('Fare>10 & Fare<50')
```

Out[99]:

	PassengerId	Survived	Pclass	Name	Sex	SibSp	Parch	Ticket
7	8	0	3	Palsson, Master. Gosta Leonard	male	3	1	349909
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	0	2	347742
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	1	0	237736
10	11	1	3	Sandstrom, Miss. Marguerite Rut	female	1	1	PP 9549
11	12	1	1	Bonnell, Miss. Elizabeth	female	0	0	113783
...	...	...	...	...	...	...	...	...
885	886	0	3	Rice, Mrs. William (Margaret Norton)	female	0	5	382652
886	887	0	2	Montvila, Rev. Juozas	male	0	0	211536
887	888	1	1	Graham, Miss. Margaret Edith	female	0	0	112053
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	1	2	W./C. 6607
889	890	1	1	Behr, Mr. Karl Howell	male	0	0	111369

394 rows × 11 columns

Step-14 Compute the sum of value

In [79]: `df['Fare'].sum()`

Out[79]: 28640.849299999998

Step-15 Compute the mean of value

```
In [83]: df['Fare'].mean()
```

Out[83]: 32.18072955056179

Step-16 Count non-null value (column)

```
In [91]: # df.count()
# df.isnull()
# df.isnull().sum()
(~df.isnull()).sum()
```

```
Out[91]: PassengerId      890
Survived      890
Pclass        890
Name          890
Sex           890
SibSp         890
Parch         890
Ticket        890
Fare          890
Cabin         203
Amount        890
dtype: int64
```

Step-17 Find Minimum or Maximum values

```
In [93]: df['Fare'].min()
```

Out[93]: 0.0

```
In [95]: df['Fare'].max()
```

Out[95]: 512.3292

Data Mining - Lab - 2

Numpy & Perform Data Exploration with Pandas

Ayushi Patel

23010101199

Numpy

1. NumPy (Numerical Python) is a powerful open-source library in Python used for numerical and scientific computing.
2. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on them efficiently.
3. NumPy is highly optimized and written in C, making it much faster than using regular Python lists for numerical operations.
4. It serves as the foundation for many other Python libraries in data science and machine learning, like pandas, TensorFlow, and scikit-learn.
5. With features like broadcasting, vectorization, and integration with C/C++ code, NumPy allows for cleaner and faster code in numerical computations.

Step 1. Import the Numpy library

```
In [3]: import numpy as np
```

Step 2. Create a 1D array of numbers

```
In [5]: arr=np.arange(10)  
arr
```

```
Out[5]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [7]: arr=np.arange(15,25)  
arr
```

```
Out[7]: array([15, 16, 17, 18, 19, 20, 21, 22, 23, 24])
```

```
In [9]: arr2=np.array([10,25,48,79,64])
arr2
```

```
Out[9]: array([10, 25, 48, 79, 64])
```

```
In [33]: arr2=np.array([10,25,48,79,64])
type(arr2)
```

```
Out[33]: numpy.ndarray
```

```
In [35]: arr2=np.array([10,25,48,79,64])
arr2.dtype
```

```
Out[35]: dtype('int32')
```

Step 3. Reshape 1D to 2D Array

```
In [31]: arr3 = np.arange(20).reshape(4,5)
arr3
```

```
Out[31]: array([[ 0,  1,  2,  3,  4],
                [ 5,  6,  7,  8,  9],
                [10, 11, 12, 13, 14],
                [15, 16, 17, 18, 19]])
```

```
In [41]: arr3=np.array([10,25,48,79,64,46,28,39,27,49,81,93,76,94,50,46,83,40,31,34])
arr3.reshape(4,5)
```

```
Out[41]: array([[10, 25, 48, 79, 64],
                [46, 28, 39, 27, 49],
                [81, 93, 76, 94, 50],
                [46, 83, 40, 31, 34]])
```

Step 4. Create a Linspace array

```
In [43]: np.linspace(14,18)
```

```
Out[43]: array([14.          , 14.08163265, 14.16326531, 14.24489796, 14.32653061,
                14.40816327, 14.48979592, 14.57142857, 14.65306122, 14.73469388,
                14.81632653, 14.89795918, 14.97959184, 15.06122449, 15.14285714,
                15.2244898 , 15.30612245, 15.3877551 , 15.46938776, 15.55102041,
                15.63265306, 15.71428571, 15.79591837, 15.87755102, 15.95918367,
                16.04081633, 16.12244898, 16.20408163, 16.28571429, 16.36734694,
                16.44897959, 16.53061224, 16.6122449 , 16.69387755, 16.7755102 ,
                16.85714286, 16.93877551, 17.02040816, 17.10204082, 17.18367347,
                17.26530612, 17.34693878, 17.42857143, 17.51020408, 17.59183673,
                17.67346939, 17.75510204, 17.83673469, 17.91836735, 18.          ])
```

```
In [45]: np.linspace(14,18,25)
```

```
Out[45]: array([14.          , 14.16666667, 14.33333333, 14.5          , 14.66666667,
               14.83333333, 15.          , 15.16666667, 15.33333333, 15.5          ,
               15.66666667, 15.83333333, 16.          , 16.16666667, 16.33333333,
               16.5          , 16.66666667, 16.83333333, 17.          , 17.16666667,
               17.33333333, 17.5          , 17.66666667, 17.83333333, 18.          ])
```

Step 5. Create a Random Numbered Array

```
In [47]: np.random.rand(8)
```

```
Out[47]: array([0.8271171 , 0.75256435, 0.08654336, 0.46993156, 0.85154777,
               0.36000452, 0.01395086, 0.76171999])
```

```
In [49]: np.random.rand(5,5)
```

```
Out[49]: array([[0.07952891, 0.07928686, 0.01711261, 0.11346888, 0.92519659],
               [0.4409731 , 0.69858814, 0.589884   , 0.56709257, 0.01403691],
               [0.97301937, 0.74918719, 0.23198325, 0.08445287, 0.74168174],
               [0.63364573, 0.85932744, 0.09333438, 0.24722792, 0.03348255],
               [0.04998347, 0.42217505, 0.718126   , 0.26142342, 0.8832608 ]])
```

Step 6. Create a Random Integer Array

```
In [53]: np.random.randint(20,40)
```

```
Out[53]: 25
```

```
In [55]: np.random.randint(20,40,size=10)
```

```
Out[55]: array([21, 28, 36, 31, 20, 29, 35, 33, 27, 25])
```

```
In [57]: np.random.randint(20,40,size=(5,5))
```

```
Out[57]: array([[27, 27, 21, 34, 32],
               [23, 26, 32, 31, 23],
               [34, 30, 24, 21, 26],
               [37, 39, 33, 38, 28],
               [27, 30, 31, 34, 37]])
```

Step 7. Create a 1D Array and get Max,Min,ArgMax,ArgMin

```
In [61]: arr4 = np.random.randint(30,60,size=10)
arr4
```

```
Out[61]: array([56, 48, 39, 49, 47, 53, 30, 43, 37, 45])
```

```
In [71]: arr4.max()
```

```
Out[71]: 56
```

```
In [63]: max(arr4)
```

```
Out[63]: 56
```

```
In [74]: arr4.min()
```

```
Out[74]: 30
```

```
In [65]: min(arr4)
```

```
Out[65]: 30
```

```
In [76]: arr4.argmax()
```

```
Out[76]: 0
```

```
In [78]: arr4.argmin()
```

```
Out[78]: 6
```

Step 8. Indexing in 1D Array

```
In [82]: arr5 = np.random.randint(30,60,size=10)
arr5
```

```
Out[82]: array([35, 40, 34, 37, 45, 34, 33, 44, 51, 57])
```

```
In [84]: arr5[5]
```

```
Out[84]: 34
```

```
In [88]: arr5[3:6]
```

```
Out[88]: array([37, 45, 34])
```

Step 9. Indexing in 2D Array

```
In [90]: arr6=np.random.randint(20,40,size=(5,5))
arr6
```

```
Out[90]: array([[27, 25, 28, 31, 21],
                [31, 37, 35, 37, 26],
                [20, 27, 33, 33, 21],
                [23, 28, 21, 21, 24],
                [38, 20, 23, 21, 25]])
```

```
In [92]: arr6[2]
```

```
Out[92]: array([20, 27, 33, 33, 21])
```

```
In [96]: arr6[1:4]
```

```
Out[96]: array([[31, 37, 35, 37, 26],
               [20, 27, 33, 33, 21],
               [23, 28, 21, 21, 24]])
```

```
In [100... # arr6[3,3]
arr6[3][3]
```

```
Out[100... 21
```

Step 10. Conditional Selection

```
In [102... arr7 = np.random.randint(20,40,size=10)
arr7
```

```
Out[102... array([31, 21, 25, 32, 23, 36, 28, 37, 30, 30])
```

```
In [113... arr7>25
```

```
Out[113... array([ True, False, False,  True, False,  True,  True,  True,  True,
                True])
```

```
In [115... arr7[arr7>25]
```

```
Out[115... array([31, 32, 36, 28, 37, 30, 30])
```

```
In [117... arr7[(arr7>25) & (arr7<35)]
```

```
Out[117... array([31, 32, 28, 30, 30])
```

🔥 You did it! 10 exercises down — you're on fire! 🔥

Pandas

Step 1. Import the necessary libraries

```
In [119... import pandas as pd
```

Step 2. Import the dataset from this [address](https://raw.githubusercontent.com/justmarkham/DAT8/master/users.csv).

Step 3. Assign it to a variable called users and use the 'user_id' as index

```
In [127... users= pd.read_csv("https://raw.githubusercontent.com/justmarkham/DAT8/master/users.csv")
```

Out[127...

	age	gender	occupation	zip_code
user_id				
1	24	M	technician	85711
2	53	F	other	94043
3	23	M	writer	32067
4	24	M	technician	43537
5	33	F	other	15213
...	...	...	...	...
939	26	F	student	33319
940	32	M	administrator	02215
941	20	M	student	97229
942	48	F	librarian	78209
943	22	M	student	77841

943 rows × 4 columns

Step 4. See the first 25 entries

In [133...

```
users.head(25)
```

Out[133...

	age	gender	occupation	zip_code
user_id				
1	24	M	technician	85711
2	53	F	other	94043
3	23	M	writer	32067
4	24	M	technician	43537
5	33	F	other	15213
6	42	M	executive	98101
7	57	M	administrator	91344
8	36	M	administrator	05201
9	29	M	student	01002
10	53	M	lawyer	90703
11	39	F	other	30329
12	28	F	other	06405
13	47	M	educator	29206
14	45	M	scientist	55106
15	49	F	educator	97301
16	21	M	entertainment	10309
17	30	M	programmer	06355
18	35	F	other	37212
19	40	M	librarian	02138
20	42	F	homemaker	95660
21	26	M	writer	30068
22	25	M	writer	40206
23	30	F	artist	48197
24	21	F	artist	94533
25	39	M	engineer	55107

Step 5. See the last 10 entries

In [139...

```
users.tail(10)
```

```
Out[139...      age  gender  occupation  zip_code
user_id
934    61      M    engineer    22902
935    42      M      doctor    66221
936    24      M       other    32789
937    48      M    educator    98072
938    38      F   technician    55038
939    26      F      student    33319
940    32      M administrator    02215
941    20      M      student    97229
942    48      F    librarian    78209
943    22      M      student    77841
```

Step 6. What is the number of observations in the dataset?

```
In [145... users.shape[0]
```

```
Out[145... 943
```

Step 7. What is the number of columns in the dataset?

```
In [147... users.shape[1]
```

```
Out[147... 4
```

Step 8. Print the name of all the columns.

```
In [149... users.columns
```

```
Out[149... Index(['age', 'gender', 'occupation', 'zip_code'], dtype='object')
```

Step 9. How is the dataset indexed?

```
In [155... users.index
```

```
Out[155... Index([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10,
      ...
      934, 935, 936, 937, 938, 939, 940, 941, 942, 943],
      dtype='int64', name='user_id', length=943)
```

Step 10. What is the data type of each column?

```
In [153...] users.dtypes
```

```
Out[153...] age          int64
gender         object
occupation     object
zip_code       object
dtype: object
```

Step 11. Print only the occupation column

```
In [161...] users['occupation']
# users.occupation
```

```
Out[161...] user_id
1          technician
2             other
3             writer
4          technician
5             other
...
939         student
940 administrator
941         student
942         librarian
943         student
Name: occupation, Length: 943, dtype: object
```

Step 12. How many different occupations are in this dataset?

```
In [163...] users['occupation'].nunique()
```

```
Out[163...] 21
```

```
In [165...] users['occupation'].unique()
```

```
Out[165...] array(['technician', 'other', 'writer', 'executive', 'administrator',
        'student', 'lawyer', 'educator', 'scientist', 'entertainment',
        'programmer', 'librarian', 'homemaker', 'artist', 'engineer',
        'marketing', 'none', 'healthcare', 'retired', 'salesman', 'doctor'],
        dtype=object)
```

Step 13. What is the most frequent occupation?

```
In [177...] users['occupation'].value_counts()
```

```
Out[177... occupation
student      196
other        105
educator     95
administrator 79
engineer     67
programmer   66
librarian    51
writer       45
executive    32
scientist    31
artist       28
technician   27
marketing    26
entertainment 18
healthcare   16
retired      14
lawyer       12
salesman     12
none         9
homemaker    7
doctor       7
Name: count, dtype: int64
```

```
In [179... users['occupation'].value_counts().head(1)
```

```
Out[179... occupation
student      196
Name: count, dtype: int64
```

```
In [185... users['occupation'].value_counts().idxmax()
```

```
Out[185... 'student'
```

Step 14. Summarize the DataFrame.

```
In [187... users.describe()
```

```
Out[187...      age
count  943.000000
mean   34.051962
std    12.192740
min     7.000000
25%    25.000000
50%    31.000000
75%    43.000000
max    73.000000
```

Step 15. Summarize all the columns

```
In [193...] users.describe(include='all')
```

```
Out[193...]      age  gender  occupation  zip_code
```

	age	gender	occupation	zip_code
count	943.000000	943	943	943
unique	NaN	2	21	795
top	NaN	M	student	55414
freq	NaN	670	196	9
mean	34.051962	NaN	NaN	NaN
std	12.192740	NaN	NaN	NaN
min	7.000000	NaN	NaN	NaN
25%	25.000000	NaN	NaN	NaN
50%	31.000000	NaN	NaN	NaN
75%	43.000000	NaN	NaN	NaN
max	73.000000	NaN	NaN	NaN

Step 16. Summarize only the occupation column

```
In [191...] users['occupation'].describe()
```

```
Out[191...] count      943
unique        21
top      student
freq         196
Name: occupation, dtype: object
```

Step 17. What is the mean age of users?

```
In [195...] users['age'].mean()
```

```
Out[195...] 34.05196182396607
```

Step 18. What is the age with least occurrence?

```
In [197...] users['age'].value_counts().tail()
```

```
Out[197... age
7      1
66     1
11     1
10     1
73     1
Name: count, dtype: int64
```

```
In [199... users['age'].value_counts().idxmin()
```

```
Out[199... 7
```

You're not just learning, you're mastering it. Keep aiming higher! 

```
In [ ]:
```

This notebook was converted with convert.ploomber.io

Data Mining

Lab - 3

Ayushi Patel

23010101199

1) First, you need to read the titanic dataset from local disk and display first five records

```
In [3]: import pandas as pd
```

```
In [5]: df=pd.read_csv('titanic.csv')
```

```
In [5]: df.head(5)
```

```
Out[5]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Tic
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	21
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/ 3101
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373

2) Identify Nominal, Ordinal, Binary and Numeric attributes from data sets and display all values.

```
In [57]: numeric=['PassengerId', 'Age', 'SibSp', 'Parch', 'Fare',]
         binary=['Survived', 'Sex']
         ordinal=['Pclass']
         nominal=['Name', 'Ticket', 'Cabin', 'Embarked']
         print(numeric, binary, ordinal, nominal )

['PassengerId', 'Age', 'SibSp', 'Parch', 'Fare'] ['Survived', 'Sex'] ['Pclass'] ['Name', 'Ticket', 'Cabin', 'Embarked']
```

```
In [16]: df['Survived'].nunique()
         df['Survived'].unique()
```

```
Out[16]: array([0, 1], dtype=int64)
```

```
In [11]: df['Pclass'].nunique()
         df['Pclass'].unique()
```

```
Out[11]: array([3, 1, 2], dtype=int64)
```

```
In [25]: df['Name'].nunique()
         # df['Name'].unique()
```

```
Out[25]: 891
```

```
In [27]: df['Sex'].nunique()
```

```
df['Sex'].unique()
```

```
Out[27]: array(['male', 'female'], dtype=object)
```

```
In [29]: df['Age'].nunique()  
df['Age'].unique()
```

```
Out[29]: array([22. , 38. , 26. , 35. , nan, 54. , 2. , 27. , 14. ,  
              4. , 58. , 20. , 39. , 55. , 31. , 34. , 15. , 28. ,  
              8. , 19. , 40. , 66. , 42. , 21. , 18. , 3. , 7. ,  
              49. , 29. , 65. , 28.5, 5. , 11. , 45. , 17. , 32. ,  
              16. , 25. , 0.83, 30. , 33. , 23. , 24. , 46. , 59. ,  
              71. , 37. , 47. , 14.5, 70.5, 32.5, 12. , 9. , 36.5 ,  
              51. , 55.5, 40.5, 44. , 1. , 61. , 56. , 50. , 36. ,  
              45.5, 20.5, 62. , 41. , 52. , 63. , 23.5, 0.92, 43. ,  
              60. , 10. , 64. , 13. , 48. , 0.75, 53. , 57. , 80. ,  
              70. , 24.5, 6. , 0.67, 30.5 , 0.42, 34.5 , 74. ])
```

```
In [31]: df['SibSp'].nunique()  
df['SibSp'].unique()
```

```
Out[31]: array([1, 0, 3, 4, 2, 5, 8], dtype=int64)
```

```
In [33]: df['Parch'].nunique()  
df['Parch'].unique()
```

```
Out[33]: array([0, 1, 2, 5, 3, 4, 6], dtype=int64)
```

```
In [37]: df['Ticket'].nunique()  
# df['Ticket'].unique()
```

```
Out[37]: 681
```

```
In [41]: df['Fare'].nunique()  
# df['Fare'].unique()
```

```
Out[41]: 248
```

```
In [45]: df['Cabin'].nunique()  
# df['Cabin'].unique()
```

```
Out[45]: 147
```

```
In [47]: df['Embarked'].nunique()  
df['Embarked'].unique()
```

```
Out[47]: array(['S', 'C', 'Q', nan], dtype=object)
```

3) Identify symmetric and asymmetric binary attributes from data sets and display all values.

```
In [7]: df['Survived'].value_counts()  
df['Sex'].value_counts()
```

```
symmetric='Sex'  
asymmetric='Survived'
```

4) For each quantitative attribute, calculate its average, standard deviation, minimum, mode, range and maximum values.

```
In [112... print('PassengerId: ')  
mean=df['PassengerId'].mean()  
print(f'\tMean = {mean}')
```

```
sd=df['PassengerId'].std()  
print(f'\tStandard deviation = {sd}')
```

```
min=df['PassengerId'].min()  
print(f'\tMinimum = {min}')
```

```
max=df['PassengerId'].max()  
print(f'\tMaximum = {max}')
```

```
mode=df['PassengerId'].mode()[0]  
print(f'\tMode = {mode}')
```

```
range= max-min  
print(f'\tRange = {range}')
```

```
PassengerId:  
    Mean = 446.0  
    Standard deviation = 257.3538420152301  
    Minimum = 1  
    Maximum = 891  
    Mode = 1  
    Range = 890
```

```
In [124... data=['PassengerId', 'Survived', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare']  
for col in data:  
    print(f"{col} :")  
    print("\tMean = ",df[col].mean())  
    print("\tStandard Deviation = ",df[col].std())  
    print("\tMinimum = ",df[col].min())  
    print("\tMaximum = ",df[col].max())  
    print("\tMode = ",df[col].mode()[0])  
    print("\tRange = ",df[col].max()-df[col].min())
```

```

PassengerId :
    Mean = 446.0
    Standard Deviation = 257.3538420152301
    Minimum = 1
    Maximum = 891
    Mode = 1
    Range = 890
Survived :
    Mean = 0.3838383838383838
    Standard Deviation = 0.4865924542648585
    Minimum = 0
    Maximum = 1
    Mode = 0
    Range = 1
Pclass :
    Mean = 2.308641975308642
    Standard Deviation = 0.8360712409770513
    Minimum = 1
    Maximum = 3
    Mode = 3
    Range = 2
Age :
    Mean = 29.69911764705882
    Standard Deviation = 14.526497332334044
    Minimum = 0.42
    Maximum = 80.0
    Mode = 24.0
    Range = 79.58
SibSp :
    Mean = 0.5230078563411896
    Standard Deviation = 1.1027434322934275
    Minimum = 0
    Maximum = 8
    Mode = 0
    Range = 8
Parch :
    Mean = 0.38159371492704824
    Standard Deviation = 0.8060572211299559
    Minimum = 0
    Maximum = 6
    Mode = 0
    Range = 6
Fare :
    Mean = 32.204207968574636
    Standard Deviation = 49.693428597180905
    Minimum = 0.0
    Maximum = 512.3292
    Mode = 8.05
    Range = 512.3292

```

6) For the qualitative attribute (class), count the frequency for each of its distinct values.

```
In [133... df['Pclass'].value_counts()
```

```
Out[133... Pclass
3      491
1      216
2      184
Name: count, dtype: int64
```

7) It is also possible to display the summary for all the attributes simultaneously in a table using the describe() function. If an attribute is quantitative, it will display its mean, standard deviation and various quantiles (including minimum, median, and maximum) values. If an attribute is qualitative, it will display its number of unique values and the top (most frequent) values.

```
In [143... # df.describe()
df.describe(include='all')
# df['Fare'].describe() -> for particular column
# df.describe(include='object') ->for particular datatype
```

```
Out[143...
```

	PassengerId	Survived	Pclass	Name	Sex	Age	S
count	891.000000	891.000000	891.000000	891	891	714.000000	891.00
unique	NaN	NaN	NaN	891	2	NaN	
top	NaN	NaN	NaN	Braund, Mr. Owen Harris	male	NaN	
freq	NaN	NaN	NaN	1	577	NaN	
mean	446.000000	0.383838	2.308642	NaN	NaN	29.699118	0.52
std	257.353842	0.486592	0.836071	NaN	NaN	14.526497	1.10
min	1.000000	0.000000	1.000000	NaN	NaN	0.420000	0.00
25%	223.500000	0.000000	2.000000	NaN	NaN	20.125000	0.00
50%	446.000000	0.000000	3.000000	NaN	NaN	28.000000	0.00
75%	668.500000	1.000000	3.000000	NaN	NaN	38.000000	1.00
max	891.000000	1.000000	3.000000	NaN	NaN	80.000000	8.00

8) For multivariate statistics, you can compute the covariance and correlation between pairs of attributes.

```
In [149... df.cov(numeric_only='true')
```

Out[149...

	PassengerId	Survived	Pclass	Age	SibSp	P
PassengerId	66231.000000	-0.626966	-7.561798	138.696504	-16.325843	-0.34
Survived	-0.626966	0.236772	-0.137703	-0.551296	-0.018954	0.03
Pclass	-7.561798	-0.137703	0.699015	-4.496004	0.076599	0.01
Age	138.696504	-0.551296	-4.496004	211.019125	-4.163334	-2.34
SibSp	-16.325843	-0.018954	0.076599	-4.163334	1.216043	0.36
Parch	-0.342697	0.032017	0.012429	-2.344191	0.368739	0.64
Fare	161.883369	6.221787	-22.830196	73.849030	8.748734	8.66

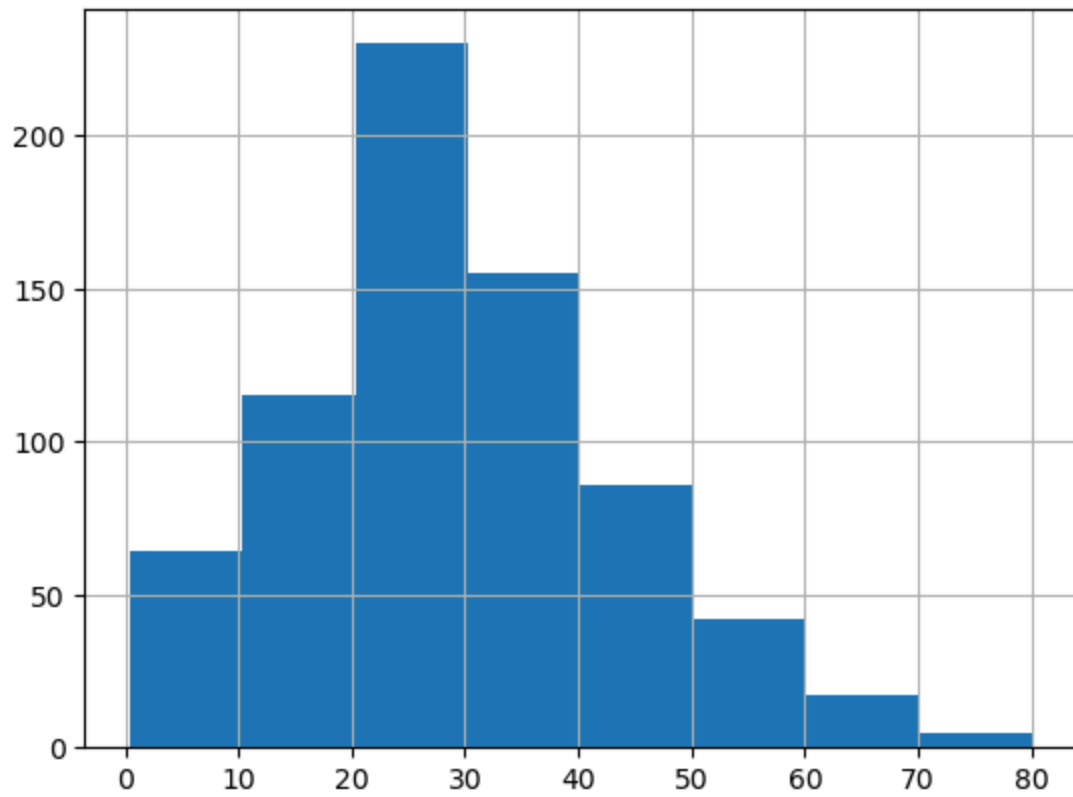
In [147... `df.corr(numeric_only='true')`

Out[147...

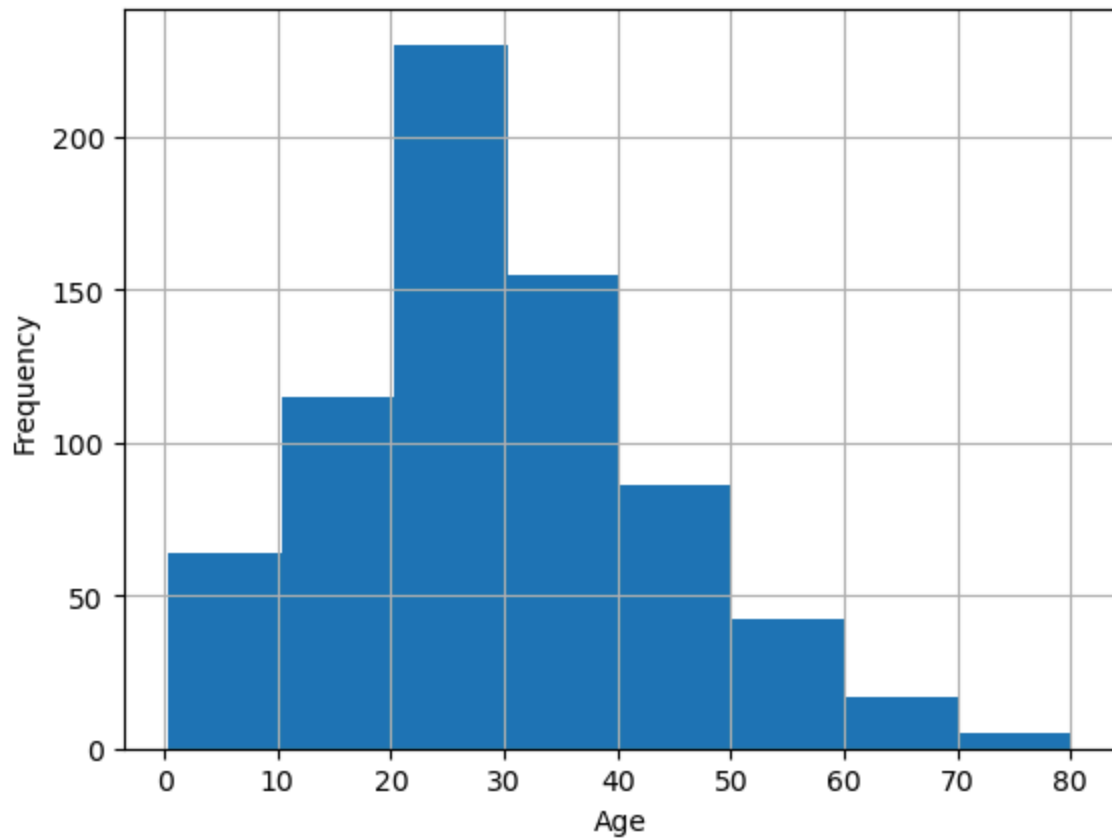
	PassengerId	Survived	Pclass	Age	SibSp	Parch
PassengerId	1.000000	-0.005007	-0.035144	0.036847	-0.057527	-0.001652
Survived	-0.005007	1.000000	-0.338481	-0.077221	-0.035322	0.081629
Pclass	-0.035144	-0.338481	1.000000	-0.369226	0.083081	0.018443
Age	0.036847	-0.077221	-0.369226	1.000000	-0.308247	-0.189119
SibSp	-0.057527	-0.035322	0.083081	-0.308247	1.000000	0.414838
Parch	-0.001652	0.081629	0.018443	-0.189119	0.414838	1.000000
Fare	0.012658	0.257307	-0.549500	0.096067	0.159651	0.216225

9) Display the histogram for Age attribute by discretizing it into 8 separate bins and counting the frequency for each bin.

In [5]: `import matplotlib.pyplot as plt`In [153... `df['Age'].hist(bins=8)`Out[153... `<Axes: >`



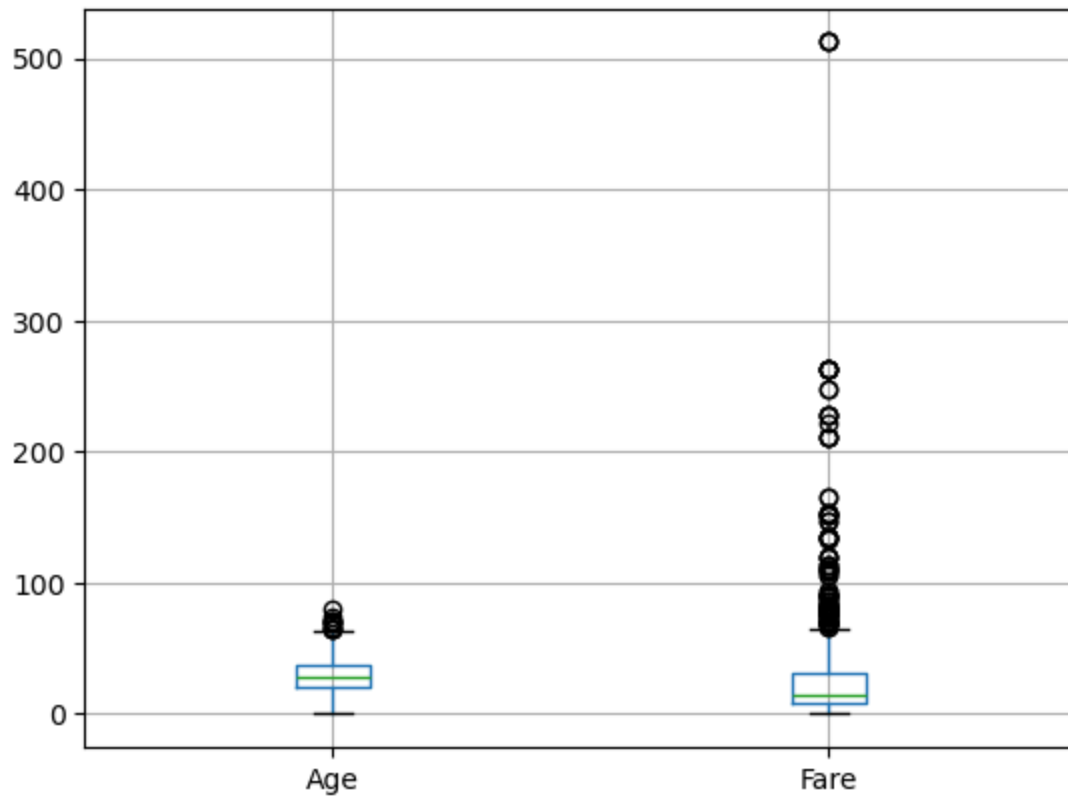
```
In [159... plt.hist(df['Age'],bins=8)
plt.grid(True)
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```



10) A boxplot can also be used to show the distribution of values for each attribute.

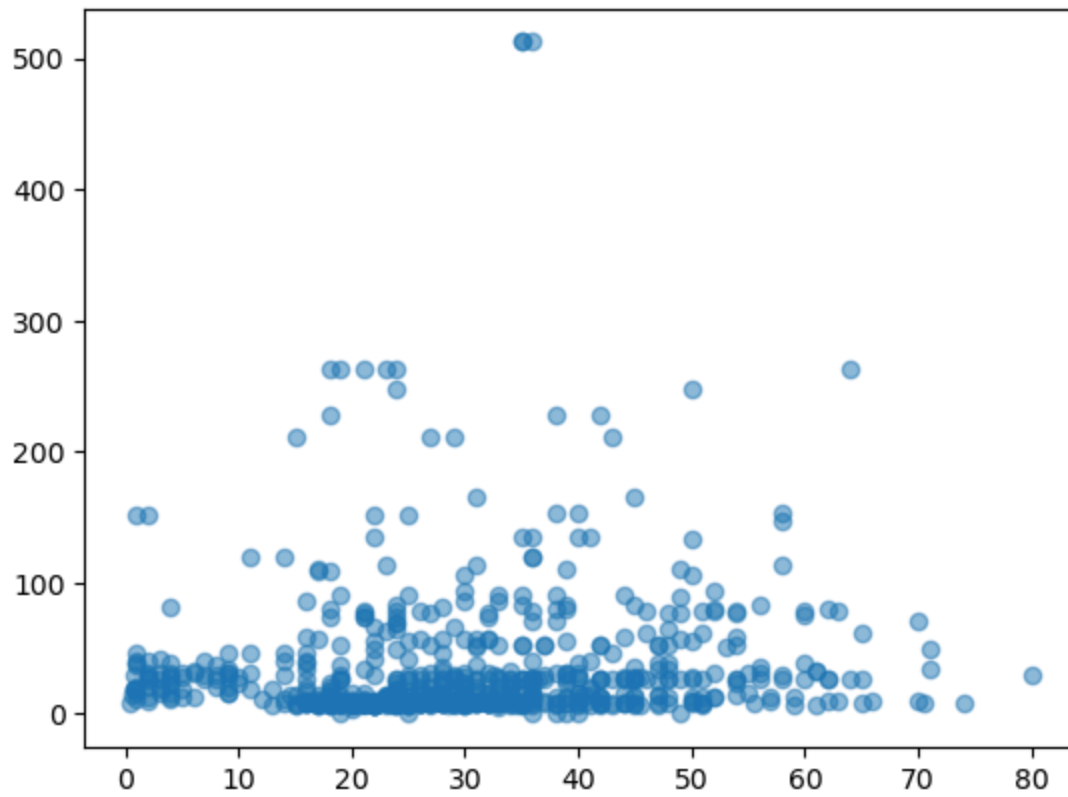
```
In [165... df[['Age', 'Fare']].boxplot()
```

```
Out[165... <Axes: >
```

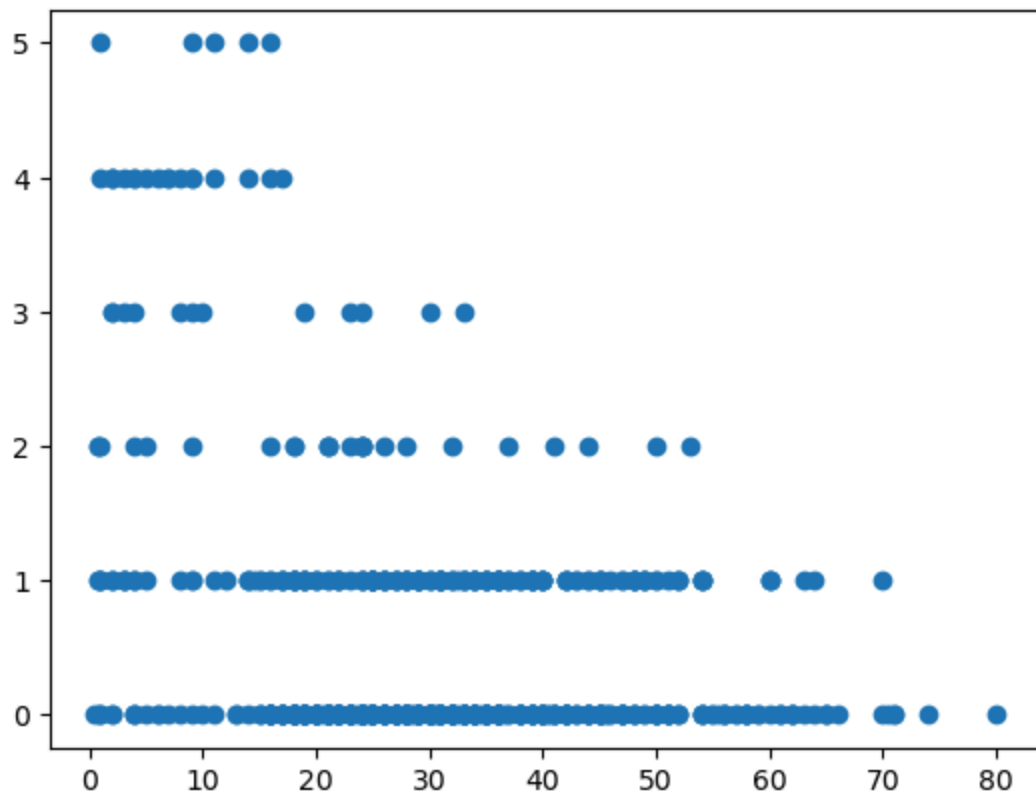


11) Display scatter plot for any 5 pair of attributes , we can use a scatter plot to visualize their joint distribution.

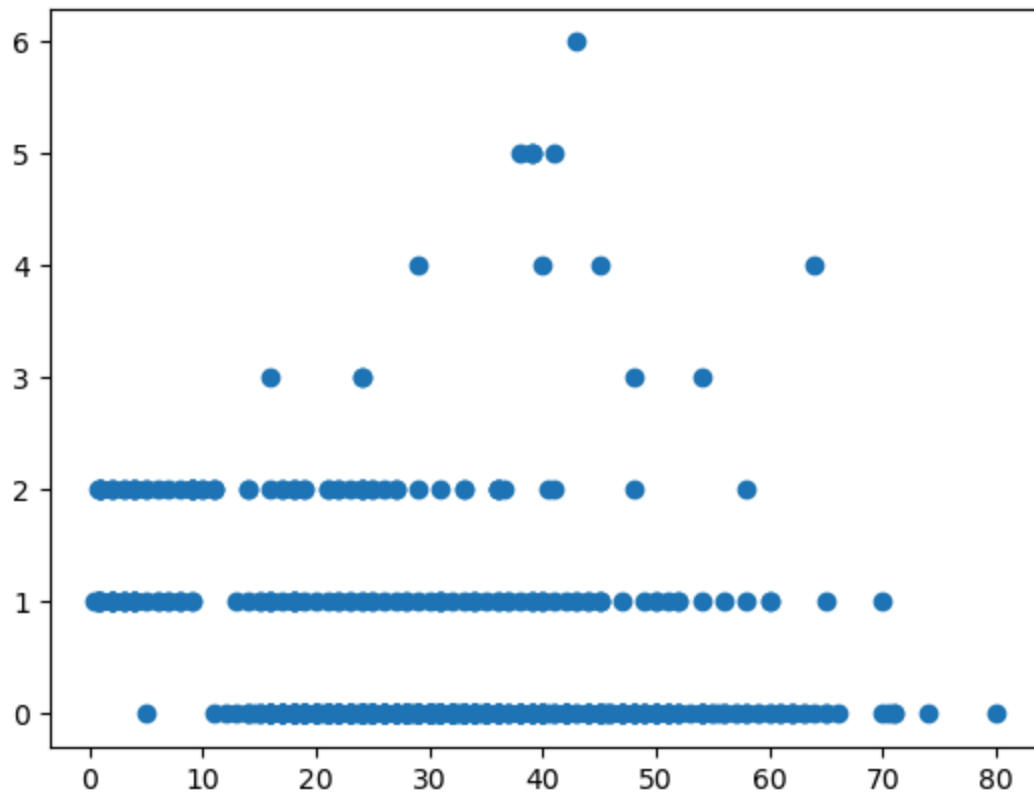
```
In [169... plt.scatter(df['Age'],df['Fare'],alpha=0.5)  
plt.show()
```



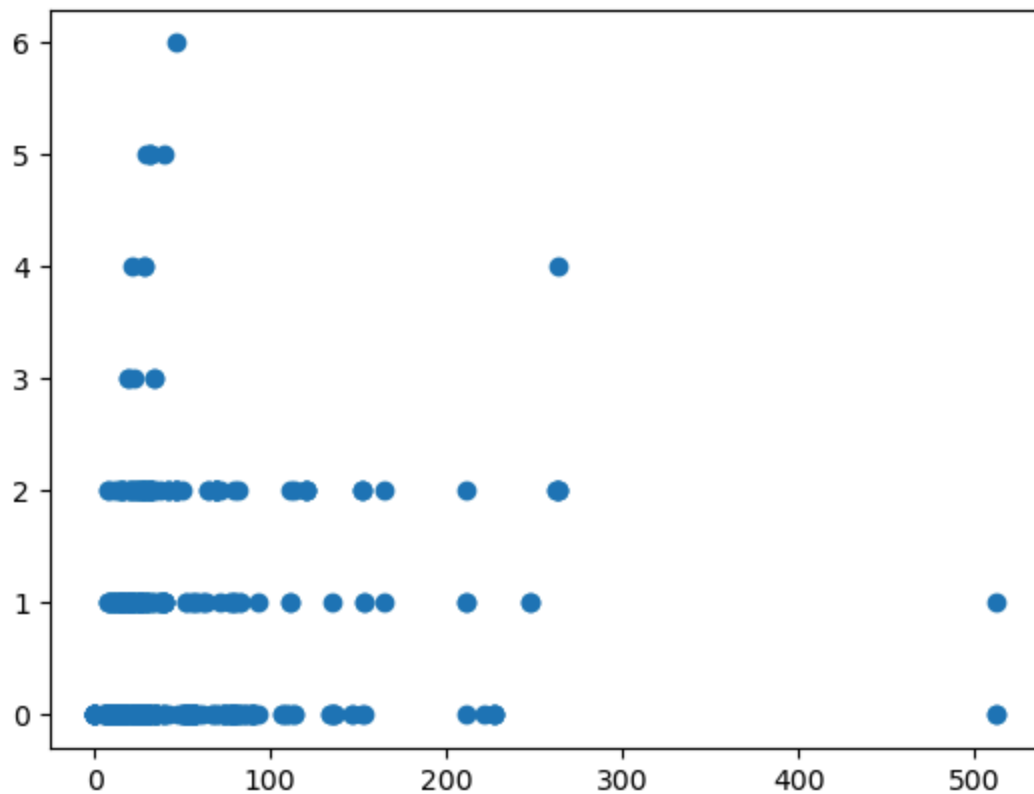
```
In [173... plt.scatter(df['Age'],df['SibSp'])
plt.show()
```



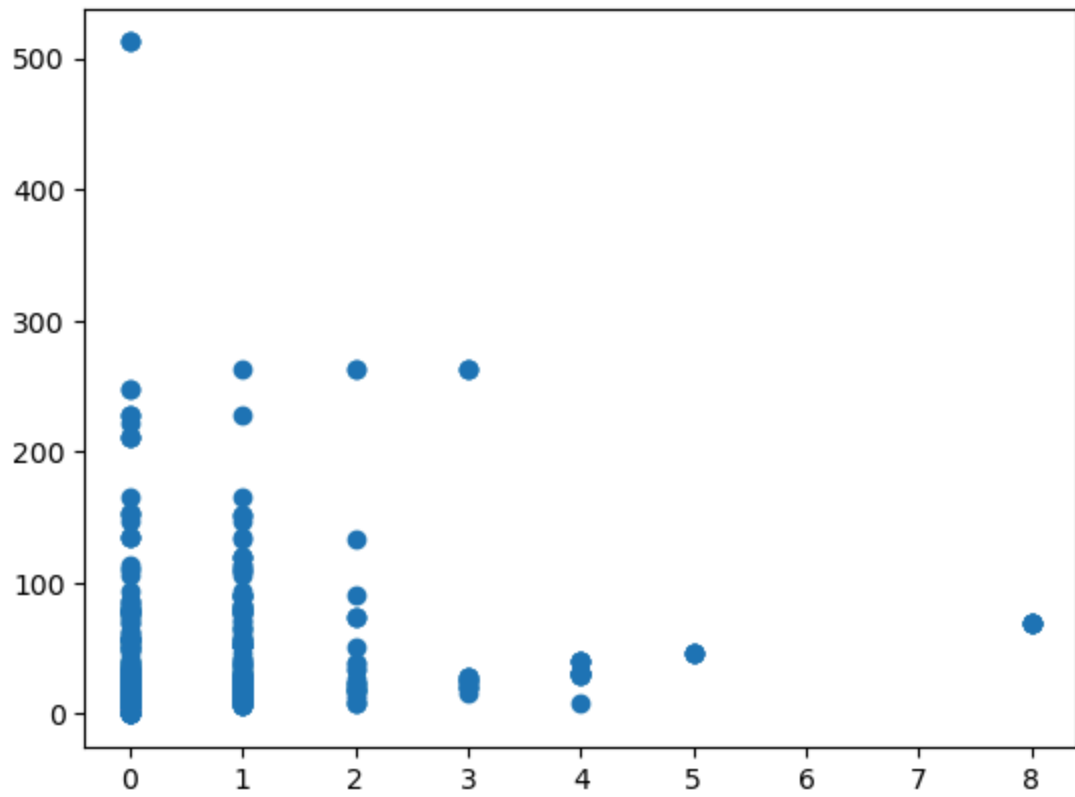
```
In [175... plt.scatter(df['Age'],df['Parch'])
plt.show()
```



```
In [15]: plt.scatter(df['Fare'],df['Parch'])
plt.show()
```



```
In [19]: plt.scatter(df['SibSp'],df['Fare'])
plt.show()
```



Data Mining

Lab - 4

Ayushi Patel

23010101199

Step 1. Import the necessary libraries

```
In [1]: import pandas as pd
```

Step 2. Import the dataset from this [address](https://raw.githubusercontent.com/justmarkham/DAT8/master/chipo.csv).

```
In [7]: chipo = pd.read_csv('https://raw.githubusercontent.com/justmarkham/DAT8/master/chipo.csv')
```

Step 3. Assign it to a variable called chipo.

```
In [9]: chipo
```

Out[9]:

	order_id	quantity	item_name	choice_description	item_price
0	1	1	Chips and Fresh Tomato Salsa	NaN	\$2.39
1	1	1	Izze	[Clementine]	\$3.39
2	1	1	Nantucket Nectar	[Apple]	\$3.39
3	1	1	Chips and Tomatillo-Green Chili Salsa	NaN	\$2.39
4	2	2	Chicken Bowl	[Tomatillo-Red Chili Salsa (Hot), [Black Beans...	\$16.98
...	...	...	...	...	...
4617	1833	1	Steak Burrito	[Fresh Tomato Salsa, [Rice, Black Beans, Sour ...	\$11.75
4618	1833	1	Steak Burrito	[Fresh Tomato Salsa, [Rice, Sour Cream, Cheese...	\$11.75
4619	1834	1	Chicken Salad Bowl	[Fresh Tomato Salsa, [Fajita Vegetables, Pinto...	\$11.25
4620	1834	1	Chicken Salad Bowl	[Fresh Tomato Salsa, [Fajita Vegetables, Lettu...	\$8.75
4621	1834	1	Chicken Salad Bowl	[Fresh Tomato Salsa, [Fajita Vegetables, Pinto...	\$8.75

4622 rows × 5 columns

Step 4. See the first 10 entries

```
In [11]: chipo.head(10)
```

Out[11]:

	order_id	quantity	item_name	choice_description	item_price
0	1	1	Chips and Fresh Tomato Salsa	NaN	\$2.39
1	1	1	Izze	[Clementine]	\$3.39
2	1	1	Nantucket Nectar	[Apple]	\$3.39
3	1	1	Chips and Tomatillo-Green Chili Salsa	NaN	\$2.39
4	2	2	Chicken Bowl	[Tomatillo-Red Chili Salsa (Hot), [Black Beans...	\$16.98
5	3	1	Chicken Bowl	[Fresh Tomato Salsa (Mild), [Rice, Cheese, Sou...	\$10.98
6	3	1	Side of Chips	NaN	\$1.69
7	4	1	Steak Burrito	[Tomatillo Red Chili Salsa, [Fajita Vegetables...	\$11.75
8	4	1	Steak Soft Tacos	[Tomatillo Green Chili Salsa, [Pinto Beans, Ch...	\$9.25
9	5	1	Steak Burrito	[Fresh Tomato Salsa, [Rice, Black Beans, Pinto...	\$9.25

Step 5. What is the number of observations in the dataset?

In [15]: `chipo.shape[0]`

Out[15]: 4622

In [13]: `chipo.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4622 entries, 0 to 4621
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   order_id              4622 non-null   int64
1   quantity              4622 non-null   int64
2   item_name             4622 non-null   object
3   choice_description     3376 non-null   object
4   item_price            4622 non-null   object
dtypes: int64(2), object(3)
memory usage: 180.7+ KB
```

Step 6. What is the number of columns in the dataset?

In [17]: `chipo.shape[1]`

Out[17]: 5

Step 7. Print the name of all the columns.

```
In [19]: chipo.columns
```

```
Out[19]: Index(['order_id', 'quantity', 'item_name', 'choice_description',  
              'item_price'],  
              dtype='object')
```

Step 8. How is the dataset indexed?

```
In [21]: chipo.index
```

```
Out[21]: RangeIndex(start=0, stop=4622, step=1)
```

Step 9. Number of Unique Items ?

```
In [31]: chipo['item_name'].nunique()
```

```
Out[31]: 50
```

Step 10. Which was the most-ordered item?

```
In [39]: c=chipo.groupby('item_name')  
         c1=c.sum()  
         c1.sort_values(['quantity'],ascending=False).head(1)  
         # c2[['order_id','quantity']].head(1) store above value in c2 and if we want  
         # c.get_group('Chicken Bowl') show items of chicken bowl only
```

```
Out[39]:
```

	order_id	quantity	choice_description	item_price
item_name				
Chicken Bowl	713926	761	[Tomatillo-Red Chili Salsa (Hot), [Black Beans...	16.9810.98 11.25 8.75 8.4911.25 \$8.75 ...

Step 11. How many items were orderd in total?

```
In [42]: chipo['quantity'].sum()
```

```
Out[42]: 4972
```

Step 12. Turn the item price into a float

Step 12.a. Check the item price type

```
In [44]: chipo['item_price'].dtypes
```

```
Out[44]: dtype('O')
```

Step 12.b. Create a lambda function and change the type of item price

```
In [72]: a="$2.34"  
print(type(a))
```

```
<class 'str'>
```

```
In [74]: x=a[1:]  
print(x)
```

```
2.34
```

```
In [76]: y=float(x)  
print(y)  
print(type(y))
```

```
2.34
```

```
<class 'float'>
```

```
In [80]: chipo['item_price'] = chipo['item_price'].apply(lambda x:float(x[1:]))  
# run only for one time only
```

Step 12.c. Check the item price type

```
In [82]: chipo.item_price.dtype
```

```
Out[82]: dtype('float64')
```

Step 14. How much was the revenue for the period in the dataset?

```
In [84]: chipo['revenue'] = chipo['quantity']*chipo['item_price']  
revenue = chipo['revenue'].sum()  
print("Revenue was: $",revenue)
```

```
Revenue was: $ 39237.02
```

Step 15. How many orders were made ?

```
In [50]: chipo['order_id'].nunique()
```

```
Out[50]: 1834
```

Step 17. How many different choice descriptions are there?

```
In [52]: chipo['choice_description'].nunique()
```

```
Out[52]: 1043
```

Step 18. What items have been ordered more than 100 times?

```
In [106... a=chipo.groupby('item_name')['quantity'].sum()  
a[a>100]
```

```
Out[106... item_name  
Bottled Water                211  
Canned Soda                  126  
Canned Soft Drink           351  
Chicken Bowl                 761  
Chicken Burrito              591  
Chicken Salad Bowl          123  
Chicken Soft Tacos           120  
Chips                        230  
Chips and Fresh Tomato Salsa 130  
Chips and Guacamole          506  
Side of Chips                110  
Steak Bowl                   221  
Steak Burrito                386  
Name: quantity, dtype: int64
```

Step 19. What is the average revenue amount per order?

```
In [112... # Solution 1  
c=chipo.groupby('order_id')['revenue'].sum().mean()  
c
```

```
Out[112... 21.39423118865867
```

```
In [114... # Solution 2  
c=chipo['revenue'].sum()/chipo['order_id'].nunique()  
c
```

```
Out[114... 21.39423118865867
```

Lab - 5 - Data Preprocessing

Ayushi Patel

23010101199

1) First, you need to read the titanic dataset from local disk and display Last five records

```
In [121]: import numpy as np
import pandas as pd
```

```
In [123]: df = pd.read_csv("titanic.csv")
df.tail()
```

```
Out[123]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211506
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112359
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W 661
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376

2) Handle Missing Values in data set [use dropna(), fillna(), and interpolate]

```
In [45]: data = df.dropna(how="any",axis=1) # deletes the columns having any single data
```

Out[45]:

	PassengerId	Survived	Pclass	Name	Sex	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	1	0	A/5 21171
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	1	0	PC 17599
2	3	1	3	Heikkinen, Miss. Laina	female	0	0	STON/O2. 3101282
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	1	0	113803
4	5	0	3	Allen, Mr. William Henry	male	0	0	373450
...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	0	0	211536
887	888	1	1	Graham, Miss. Margaret Edith	female	0	0	112053
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	1	2	W./C. 6607
889	890	1	1	Behr, Mr. Karl Howell	male	0	0	111369
890	891	0	3	Dooley, Mr. Patrick	male	0	0	370376

891 rows × 9 columns

In [107... `df['Age'] = None` # to add the null value we assign it with none

In [125... `df`

Out[125...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STC 31
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	3
...	...	...	...	...	...	...	...	...	
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	2
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	1
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	1
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	3

891 rows × 12 columns

In [113...

```
data = df.dropna(how="all",axis=1) #it delete the column which contains all data
```

Out[113...

	PassengerId	Survived	Pclass	Name	Sex	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	1	0	A/5 21171
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	1	0	PC 17599
2	3	1	3	Heikkinen, Miss. Laina	female	0	0	STON/O2. 3101282
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	1	0	113803
4	5	0	3	Allen, Mr. William Henry	male	0	0	373450
...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	0	0	211536
887	888	1	1	Graham, Miss. Margaret Edith	female	0	0	112053
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	1	2	W./C. 6607
889	890	1	1	Behr, Mr. Karl Howell	male	0	0	111369
890	891	0	3	Dooley, Mr. Patrick	male	0	0	370376

891 rows × 11 columns

In [131...

```
#withfillna

newdata = df.copy()
data_withfillna = newdata.fillna("xyz") #fills null value with given variable
data_withfillna = newdata.fillna({'Age': 35, 'Cabin': "Not Aval"})
data_withfillna
meanAge= df.Age.mean()
meanAge
```

```

modeCabin =df.Cabin.mode()
modeCabin
modeCabin =df.Cabin.mode()[0][0:3]
modeCabin
data_withfillna = newdata.fillna({'Cabin':'B96'})
data_withfillna

```

Out[131]...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STC 31
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	3
...	...	...	...	...	...	...	...	...	
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	2
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	1
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	1
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	3

891 rows × 12 columns

```
In [117]: data_withinterpolate = newdata.interpolate()  
data_withinterpolate
```

C:\Users\ayush\AppData\Local\Temp\ipykernel_18516\519773704.py:1: FutureWarning: DataFrame.interpolate with object dtype is deprecated and will raise in a future version. Call obj.infer_objects(copy=False) before interpolating instead.
data_withinterpolate = newdata.interpolate()

Out[117]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
0	1	0	3	Braund, Mr. Owen Harris	male	None	1	0	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	None	1	0	PC
2	3	1	3	Heikkinen, Miss. Laina	female	None	0	0	ST 3:
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	None	1	0	:
4	5	0	3	Allen, Mr. William Henry	male	None	0	0	:
...	...	...	...	...	...	...	...	...	
886	887	0	2	Montvila, Rev. Juozas	male	None	0	0	:
887	888	1	1	Graham, Miss. Margaret Edith	female	None	0	0	:
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	None	1	2	
889	890	1	1	Behr, Mr. Karl Howell	male	None	0	0	:
890	891	0	3	Dooley, Mr. Patrick	male	None	0	0	:

891 rows × 12 columns

3) Apply Scaling to AGE attribute with min max, decimal scaling and z score.

```
In [101]: puredata = df.copy()
puredata['Age'] = puredata['Age'].interpolate()
puredata
```

```
Out[101]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STC 31
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	3
...	...	...	...	...	...	...	...	...	
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	2
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	1
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	22.5	1	2	
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	1
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	3

891 rows × 12 columns

```
In [103... #using min-max normalization
data_minmax = puredata.copy()
age_mm = data_minmax['Age']
age_min = age_mm.min()
age_max = age_mm.max()
data_minmax['MinMaxAge'] = (age_mm - age_min)/(age_max - age_min)
data_minmax[['Age', 'MinMaxAge']]
```

Out[103... **Age** **MinMaxAge**

0	22.0	0.271174
1	38.0	0.472229
2	26.0	0.321438
3	35.0	0.434531
4	35.0	0.434531
...	...	...
886	27.0	0.334004
887	19.0	0.233476
888	22.5	0.277457
889	26.0	0.321438
890	32.0	0.396833

891 rows × 2 columns

```
In [133... #Using decimal scaling
data_ds = puredata.copy()
age_ds = data_ds['Age']
max_age = age_ds.max()
temp = len(str(int(max_age)))
print(max_age)
print(temp)
data_ds['DecimalScaleAge'] = age_ds/(10**temp)
data_ds[['Age', 'DecimalScaleAge']]
```

80.0
2

Out[133...

	Age	DecimalScaleAge
0	22.0	0.220
1	38.0	0.380
2	26.0	0.260
3	35.0	0.350
4	35.0	0.350
...	...	...
886	27.0	0.270
887	19.0	0.190
888	22.5	0.225
889	26.0	0.260
890	32.0	0.320

891 rows × 2 columns

In [137...

```
# Z scaling
data_zs = puredata.copy()
age_zs = data_zs['Age']
age_mean = age_zs.mean()
age_std = age_zs.std()
data_zs['ZScoreAge'] = (age_zs - age_mean)/age_std
data_zs[['Age', 'ZScoreAge']]
```

Out[137...

	Age	ZScoreAge
0	22.0	-0.555738
1	38.0	0.595147
2	26.0	-0.268017
3	35.0	0.379356
4	35.0	0.379356
...	...	...
886	27.0	-0.196086
887	19.0	-0.771528
888	22.5	-0.519772
889	26.0	-0.268017
890	32.0	0.163565

891 rows × 2 columns



Darshan
UNIVERSITY

Data Mining

Lab - 6

Dimensionality Reduction using NumPy

Ayushi Patel

23010101199

What is Data Reduction?

Data reduction refers to the process of reducing the amount of data that needs to be processed and stored, while preserving the essential patterns in the data.

Why do we reduce data?

- To reduce computational cost.
- To remove noise and redundant features.
- To improve model performance and training time.
- To visualize high-dimensional data in 2D or 3D.

Common data reduction techniques include:

- Principal Component Analysis (PCA)
- Feature selection
- Sampling

What is Principal Component Analysis (PCA)?

PCA is a **dimensionality reduction technique** that transforms a dataset into a new coordinate system. It identifies the **directions (principal components)** where the variance of the data is maximized.

Key Concepts:

- **Principal Components:** New features (linear combinations of original features) capturing most variance.
- **Eigenvectors & Eigenvalues:** Used to compute these principal directions.
- **Covariance Matrix:** Measures how features vary with each other.

PCA helps in **visualizing high-dimensional data**, **noise reduction**, and **speeding up algorithms**.

NumPy Functions Summary for PCA

Function	Purpose
<code>np.mean(X, axis=0)</code>	Compute mean of each column (feature-wise mean).
<code>X - np.mean(X, axis=0)</code>	Centering the data (zero mean).
<code>np.cov(X, rowvar=False)</code>	Compute covariance matrix for features.
<code>np.linalg.eigh(cov_mat)</code>	Get eigenvalues and eigenvectors (for symmetric matrices).
<code>np.argsort(values) [::-1]</code>	Sort values in descending order.
<code>np.dot(X, eigenvectors)</code>	Project original data onto new axes.

Step 1: Load the Iris Dataset

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
In [3]: iris=pd.read_csv("iris.csv")
iris
```

Out[3]:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

```
In [11]: x=iris.drop(columns="species")
y=iris['species'].map({
    "setosa":0,
    "virginica":1,
    "versicolor":2
})
print("Dimension of iris",x.shape)
```

Dimension of iris (150, 4)

Step 2: Standardize the data (zero mean)

```
In [13]: x_meaned = x - np.mean(x,axis=0)
print("Data after centering first 5 rows:\n",x_meaned[:5])
print(x_meaned.shape)
```

Data after centering first 5 rows:

	sepal_length	sepal_width	petal_length	petal_width
0	-0.743333	0.442667	-2.358	-0.999333
1	-0.943333	-0.057333	-2.358	-0.999333
2	-1.143333	0.142667	-2.458	-0.999333
3	-1.243333	0.042667	-2.258	-0.999333
4	-0.843333	0.542667	-2.358	-0.999333

(150, 4)

Step 3: Compute the Covariance Matrix

```
In [15]: cov_mat=np.cov(x_meaned,rowvar=False)
print("Covariance matrix shape:",cov_mat.shape)
print(cov_mat)
```

```
Covariance matrix shape: (4, 4)
[[ 0.68569351 -0.042434  1.27431544  0.51627069]
 [-0.042434   0.18997942 -0.32965638 -0.12163937]
 [ 1.27431544 -0.32965638  3.11627785  1.2956094 ]
 [ 0.51627069 -0.12163937  1.2956094  0.58100626]]
```

Step 4: Compute eigenvalues and eigenvectors

```
In [17]: eigen_values, eigen_vectors = np.linalg.eigh(cov_mat)
print("Eigenvalues:\n",eigen_values)
print("Eigenvectors (first 2):\n",eigen_vectors)
```

```
Eigenvalues:
[0.02383509 0.0782095  0.24267075 4.22824171]
Eigenvectors (first 2):
[[ 0.31548719  0.58202985  0.65658877 -0.36138659]
 [-0.3197231  -0.59791083  0.73016143  0.08452251]
 [-0.47983899 -0.07623608 -0.17337266 -0.85667061]
 [ 0.75365743 -0.54583143 -0.07548102 -0.3582892 ]]
```

Step 5: Sort eigenvalues and eigenvectors in descending order

```
In [23]: sorted_index=np.argsort(eigen_values)[::-1]
sorted_eigenvalues = eigen_values[sorted_index]
sorted_eigenvectors = eigen_vectors[:,sorted_index]
print("Sort index",sorted_index)
print("Sort eigen_value",sorted_eigenvalues)
print("Sort eigen_vector",sorted_eigenvectors)
```

```
Sort index [3 2 1 0]
Sort eigen_value [4.22824171 0.24267075 0.0782095  0.02383509]
Sort eigen_vector [[-0.36138659  0.65658877  0.58202985  0.31548719]
 [ 0.08452251  0.73016143 -0.59791083 -0.3197231 ]
 [-0.85667061 -0.17337266 -0.07623608 -0.47983899]
 [-0.3582892  -0.07548102 -0.54583143  0.75365743]]
```

Step 6: Select the top k eigenvectors (top 2)

```
In [25]: k=2
eigenvector_subset = sorted_eigenvectors[:,0:k]
print(eigenvector_subset.shape)
```

(4, 2)

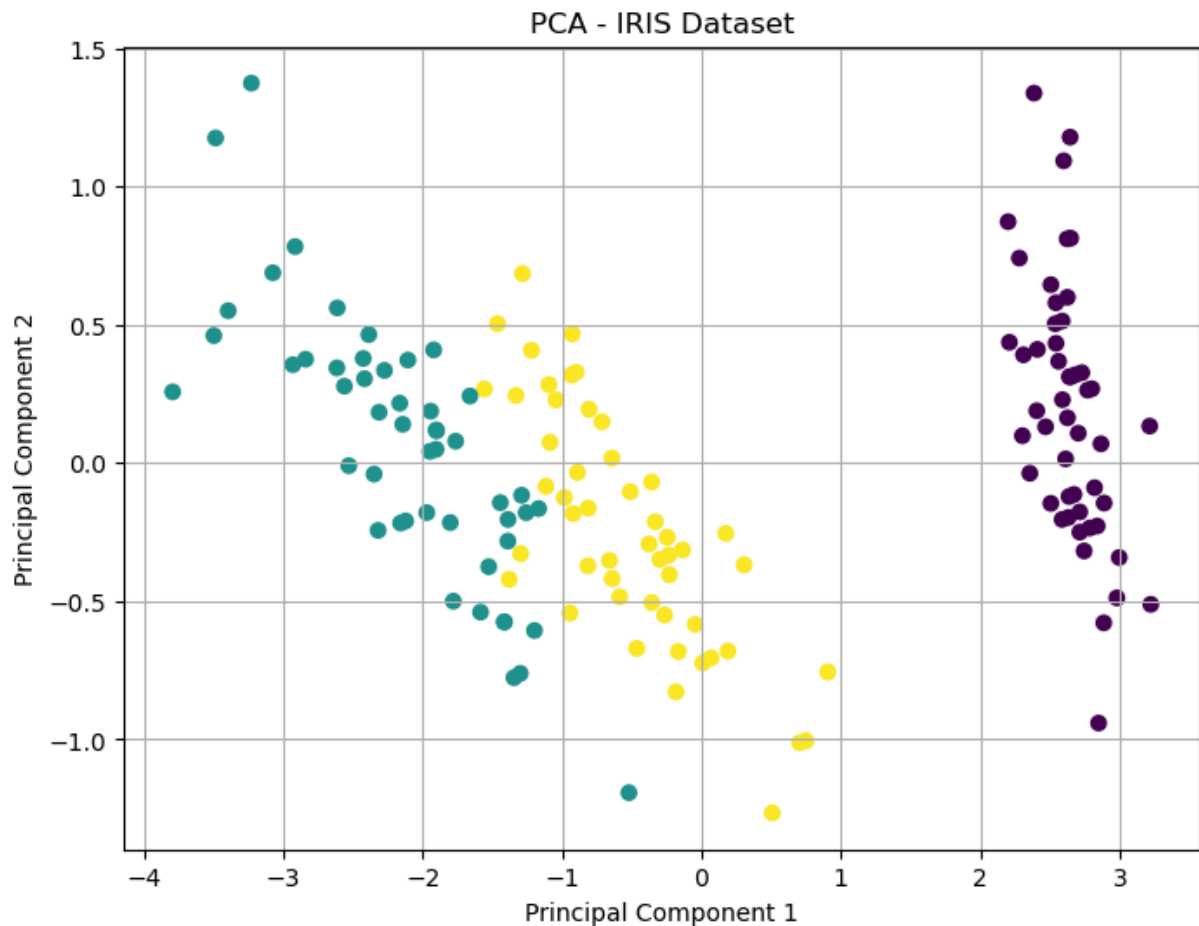
Step 7: Project the data onto the top k eigenvectors

```
In [27]: x_reduced = np.dot(x_meaned, eigenvector_subset)
print("Reduced data shape:", x_reduced.shape)
```

Reduced data shape: (150, 2)

Step 8: Plot the PCA-Reduced Data

```
In [29]: plt.figure(figsize=(8,6))
plt.scatter(x_reduced[:,0], x_reduced[:,1], c=y, cmap='viridis')
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title('PCA - IRIS Dataset')
plt.grid(True)
plt.show()
```



Extra - Bining Method

5,10,11,13,15,35,50,55,72,92,204,215.

Partition them into three bins by each of the following methods: (a) equal-frequency (equal-depth) partitioning (b) equal-width partitioning

```
In [3]: data = [5, 10, 11, 13, 15, 35, 50, 55, 72, 92, 204, 215]
data.sort()
bins = 3

print("Original Data:", data)
print("\nEqual-Frequency Binning:")

size = len(data) // bins
for i in range(0, len(data), size):
    bin_data = data[i:i + size]
    print(f"Bin {i//size + 1}: {bin_data}")

print("\nEqual-Width Binning:")

min_val = min(data)
```

```

max_val = max(data)
width = (max_val - min_val) / bins

bin1 = []
bin2 = []
bin3 = []

for num in data:
    if num < min_val + width:
        bin1.append(num)
    elif num < min_val + 2*width:
        bin2.append(num)
    else:
        bin3.append(num)

print(f"Bin 1: {bin1}")
print(f"Bin 2: {bin2}")
print(f"Bin 3: {bin3}")

```

Original Data: [5, 10, 11, 13, 15, 35, 50, 55, 72, 92, 204, 215]

Equal-Frequency Binning:

Bin 1: [5, 10, 11, 13]

Bin 2: [15, 35, 50, 55]

Bin 3: [72, 92, 204, 215]

Equal-Width Binning:

Bin 1: [5, 10, 11, 13, 15, 35, 50, 55, 72]

Bin 2: [92]

Bin 3: [204, 215]

In []:

Data Mining

Lab - 7 (Part 2)

Ayushi Patel

23010101199

Step 1: Load the Dataset

Load the `Tdata.csv` file and display the first few rows.

```
In [5]: import numpy as np
import pandas as pd
import matplotlib.pyplot as py
```

```
In [6]: data = pd.read_csv('Tdata.csv')
data
```

```
Out[6]:
```

	Transaction	bread	butter	coffee	eggs	jam	milk
0	T1	1	1	0	0	0	1
1	T2	1	1	0	0	1	0
2	T3	1	0	0	1	0	1
3	T4	1	1	0	0	0	1
4	T5	1	0	1	0	0	0
5	T6	0	0	1	1	1	0

Step 2: Drop the 'Transaction' Column

We're only interested in the items (not the transaction IDs).

```
In [27]: data = data.drop('Transaction', axis=1)
data
```

```
Out[27]:
```

	bread	butter	coffee	eggs	jam	milk
0	1	1	0	0	0	1
1	1	1	0	0	1	0
2	1	0	0	1	0	1
3	1	1	0	0	0	1
4	1	0	1	0	0	0
5	0	0	1	1	1	0

Step 3: Count Single Items

See how many transactions include each item.

```
In [29]: data.sum()
```

```
Out[29]: bread      5
butter      3
coffee     2
eggs        2
jam         2
milk        3
dtype: int64
```

Step 4: Define Apriori Function

This function finds frequent itemsets of size 1, 2, and 3 with minimum support.

```
In [57]: from itertools import combinations

def find_frequent_itemsets(data,min_support):
    n = len(data)
    result = []

    for k in [1,2,3]:
        for items in combinations(data.columns,k):
            mask = data[list(items)].all(axis=1)
            support = mask.sum()/n
            # print(f"{frozenset(items)}->{round(support,2)}")
            if support >= min_support:
                result.append((frozenset(items), round(support,2)))

    return result

# find_frequent_itemsets(data,2)
```

Step 5: Run Apriori

Set `min_support = 0.6` and display the frequent itemsets.

```
In [59]: frequent_itemsets = find_frequent_itemsets(data,min_support=0.6)

for itemset, support in frequent_itemsets:
    print(f"{set(itemset)} -> support: {support}")

{'bread'} -> support: 0.83
```

Step 6 Display as a DataFrame

```
In [61]: result_df = pd.DataFrame(frequent_itemsets, columns=['Itemset','Support'])
result_df
```

```
Out[61]:
```

	Itemset	Support
0	(bread)	0.83

Orange Tool : - >Generate Same Frequent Patterns in Orange tools

```
In [ ]:
```

Extra : - > Define Apriori Function without itertools

```
In [9]: def find_frequent_itemsets_no_itertools(data, min_support):
    n = len(data)
    result = []

    # For k = 1
    for i in range(len(data.columns)):
        items = (data.columns[i],)
        mask = data[list(items)].all(axis=1)
        support = mask.sum() / n
        if support >= min_support:
            result.append((frozenset(items), round(support, 2)))

    # For k = 2
    for i in range(len(data.columns)):
        for j in range(i + 1, len(data.columns)):
            items = (data.columns[i], data.columns[j])
            mask = data[list(items)].all(axis=1)
            support = mask.sum() / n
            if support >= min_support:
                result.append((frozenset(items), round(support, 2)))

    # For k = 3
    for i in range(len(data.columns)):
        for j in range(i + 1, len(data.columns)):
            for k in range(j + 1, len(data.columns)):
                items = (data.columns[i], data.columns[j], data.columns[k])
                mask = data[list(items)].all(axis=1)
                support = mask.sum() / n
                if support >= min_support:
                    result.append((frozenset(items), round(support, 2)))

    return result
```

```
In [11]: frequent_itemsets = find_frequent_itemsets_no_itertools(data, min_support=0.

# Display the output
for items, support in frequent_itemsets:
    print(f"{set(items)} -> {support}")
```

```
{'Transaction'} -> 1.0
{'bread'} -> 0.83
{'butter'} -> 0.5
{'milk'} -> 0.5
{'bread', 'Transaction'} -> 0.83
{'Transaction', 'butter'} -> 0.5
{'milk', 'Transaction'} -> 0.5
{'bread', 'butter'} -> 0.5
{'bread', 'milk'} -> 0.5
{'bread', 'Transaction', 'butter'} -> 0.5
{'bread', 'milk', 'Transaction'} -> 0.5
```